

Table of Contents

Regulatory Radar — IBM Bobathon Challenge

Companies are drowning in EU product regulations that change constantly. Build an AI agent that **pulls the current rules from live sources, works out which companies in the portfolio are not compliant today, and alerts them** — with the gap, the source, the deadline, and the fix. Built with **IBM Bob**; alerts fired with **Twilio**.

GenAI Builders Day · GDGoC TUM Campus Heilbronn · partner challenge by EcoComply

 **Questions any time?** Join the event WhatsApp group:
<https://chat.whatsapp.com/BQf8Eul1t2gA7LCaBD1z2Q>

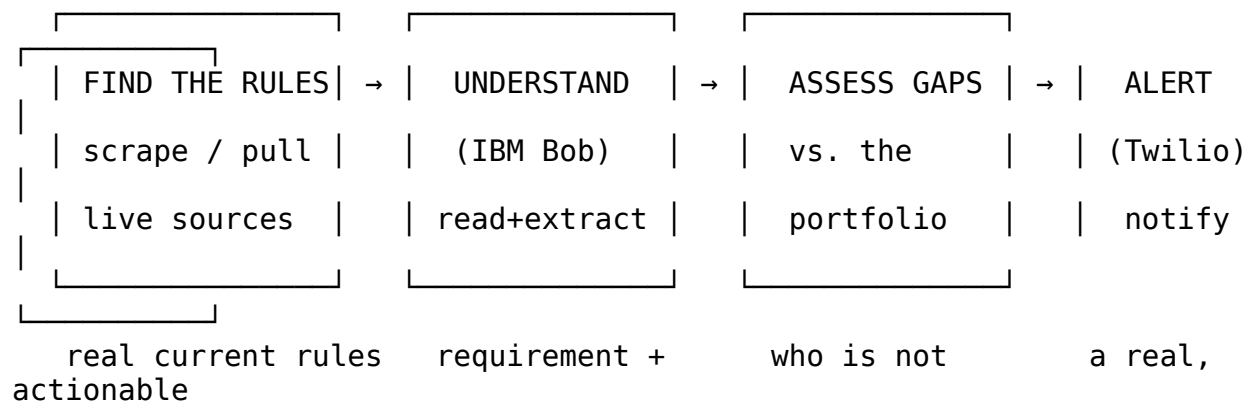
TL;DR

You're given a **portfolio of electronics SMEs** (the companies to protect). Your agent goes out to the **live web**, finds **current EU requirements** those companies must meet, figures out **where they're falling short right now**, and **sends a real alert** for each gap. Pick your regulations and depth. One real gap, from a real source, with a real alert firing, beats a big plan.

Why this matters

EcoComply is a Heilbronn startup that keeps electronics SMEs EU-market-ready. One of their services is *continuous monitoring of regulatory updates* — and a lot of it is still manual: people read legislation portals, map rules to clients by hand, and email them one by one. A missed rule means fines, blocked shipments, or delisting. **That monitor → assess → alert loop is exactly what you're going to automate.**

What you'll build



(see SOURCES.md) deadline + scope compliant today message
per company

Quickstart

```
git clone https://github.com/shubham-mw/regulatory-radar-bobathon.git
cd regulatory-radar-bobathon
python3 starter.py           # loads the portfolio, runs offline on the
                             sample feed, prints a sample gap+alert
```

Then: 1. Get your **IBM Bob** access (30-day free): <https://ibm.biz/student-bobathon> 2. Grab your **Twilio** credit — promo code **TUM-TWILIO-50** (for the alert step). 3. Open this repo in Bob and ask it to read README.md, DATASET_README.md and SOURCES.md, then help you build the pipeline (see [Working with IBM Bob](#)).

What's in the repo

File	What it is
partners.json / partners.csv	Your fixed dataset: the SME portfolio (companies + products to assess).
SOURCES.md	Start here for rules: a curated list of live regulatory sources to scrape/query.
taxonomy.json	Controlled vocabulary (product categories, substances, regulation families).
sample_expected_output.json	The shape of one finding (gap + source + alert) your agent should emit.
starter.py	A runnable scaffold: loads the portfolio, a fetch → assess → alert skeleton, offline fallback.
regulatory_updates.json + feed/	Offline sample / fallback feed — only for dev or if live scraping fails on the day.
DATASET_README.md	Full field-by-field data dictionary.

By the numbers: 22 companies · 53 products · a curated source list · an offline fallback feed. Every company and contact is **fabricated and safe** (@example.com, placeholder phones) — use your **own** Twilio test number/email for the alert demo so nothing reaches a real person.

The challenge, step by step

Step 1 — Find the current rules (live)

Pull **current** EU product requirements from real sources. **SOURCES.md** lists good starting points (EUR-Lex, ECHA, national EPR registers, EPREL, agency feeds) with tips. Scrape pages, hit official APIs/RSS, or read published lists — your call. (*Offline or stuck? Fall back to regulatory_updates.json / feed/ — but live is the real thing.*)

Step 2 — Understand each rule (use IBM Bob)

Use **IBM Bob** to read each requirement and extract the facts you need: which product categories / substances / markets it covers, the deadline, the conditions, and whether it's the current version.

Step 3 — Assess the portfolio for gaps

For each company/product in `partners.json`, decide whether a current requirement applies and whether they've met it. A **gap** = an obligation that applies and isn't satisfied. Reason from the product's attributes (category, substances, battery type, radio, markets, intended use...). A few companies carry an explicit `compliance_status` you can check against; for the rest, you infer. Mind the look-alikes (right category, wrong market/substance/attribute). **Cite the source** for every gap.

Step 4 — Alert

For each gap, send **one** clear, actionable message on the company's `preferred_channel` (email / SMS / WhatsApp): *"Your product Y must meet rule X by deadline Z — here's the fix."* **Twilio** is the quick way to fire a real notification (promo TUM-TWILIO-50). Use **your own** test number/email. At least one real alert firing in your demo is the wow moment.

Output shape: see `sample_expected_output.json` — a finding with the company, product, regulation, **source URL**, the gap, deadline, recommended action, and the alert.

Difficulty tiers (pick your level)

Tier	Who	Do this
Beginner	New to coding/AI	Use the offline sample feed + the <code>compliance_status</code> companies: find a known gap and fire one alert. The pipeline end-to-end on easy mode.
Core (target)	Most teams	Scrape one or two live sources , use Bob to extract the rule, assess several portfolio companies for gaps, fire real alerts. The full loop on live data.
Stretch	Strong teams	Cover more regulations, add a risk dashboard , de-duplicate across sources, prioritise by deadline, multi-language alerts, or an audit log of "rule → source → company → action".

A correct **Core** solution on live data beats a flashy-but-wrong Stretch one.

Working with IBM Bob

Open the repo in Bob and ask it to read `README.md`, `DATASET_README.md` and `SOURCES.md`, then scaffold the pipeline. Example prompts:

- **Orient:** “Read the three docs in this repo and summarise the four steps and the data I have.”
- **Find rules:** “Here’s the HTML of an EUR-Lex / ECHA page: `<paste>`. Extract each current requirement as JSON: regulation, what it requires, affected categories/substances/markets, deadline, and the source URL.”
- **Assess a company:** “Given this company `<paste from partners.json>` and this requirement `<paste>`, does it apply, and is there a gap? Reason through market, category, substance and attributes, and cite the deciding factor.”
- **Alert:** “Draft a concise SMS (under 300 chars) telling company X that product Y must meet rule Z by the deadline, with one recommended action and the source link.”

Tools

- **IBM Bob — required.** Build the pipeline fast *and* use it to read rules and reason about gaps. IBM mentors on-site. (30-day free: <https://ibm.biz/student-bobathon>)
- **Twilio — recommended for alerts.** Real SMS/WhatsApp/email. Promo **TUM-TWILIO-50**. Any channel that delivers a real alert counts.
- **Anything else** — any language, scraping lib, or framework. Keep the scope to a thin end-to-end slice.

What to submit

- **A working prototype** — a live demo where a current rule → a real gap → a real alert fires.
- **Your findings** — JSON in the shape of `sample_expected_output.json`, with source URLs.
- **A short README** — what it does, your stack, and how you used **IBM Bob** and **Twilio**.
- **A 3-min demo + 1-min pitch.**

How you’re judged

Criterion	What we look for	Weight
Works end-to-end	Live: a current rule → a real gap → a real alert fires	30%
Quality of insight	The gap is real and correctly reasoned, with a source cited	25%
Use of IBM Bob	How effectively Bob built and powered it	15%
Alert delivery	A real notification actually fires; sensible channel	10%
Real-world fit	Would EcoComply use it?	10%

Criterion	What we look for	Weight
	Actionable, auditable, right deadline	
Demo & communication	Clear story, crisp pitch	10%

No hidden answer key — judging is on what you build and present. Show a **real source**, a **real gap**, and a **real alert**, and explain your reasoning.

? FAQ

- **Do we have to scrape live?** That's the goal, but if a site is hard or Wi-Fi fails, fall back to `regulatory_updates.json / feed/`. Don't get stuck — a working loop matters more.
- **How do we know a company is non-compliant?** You reason from the product's attributes against the rule. A few companies have an explicit `compliance_status` to make gaps concrete; for the rest you assess.
- **Do alerts have to be real?** Aim for at least one real notification in the demo. Twilio makes it trivial.
- **Teams?** Solo or teams, formed at the event. All levels welcome — pick a tier.
- **Stuck?** Ask any mentor, or post in the WhatsApp group:
<https://chat.whatsapp.com/BQf8Eul1t2gA7LCaBD1z2Q>

Repo structure

```

regulatory-radar/
├── README.md           ← you are here (the whole challenge)
├── SOURCES.md          ← where to get current regulations
├── (start here for rules)
├── DATASET_README.md  ← full data dictionary
├── starter.py          ← runnable scaffold (offline fallback
included)
├── partners.json / .csv ← the SME portfolio (your fixed
dataset)
├── taxonomy.json       ← controlled vocabulary
├── sample_expected_output.json ← the output shape (one finding)
├── regulatory_updates.json ← offline sample / fallback feed
├── feed/               ← offline sample HTML pages
└── dataset_stats.json

```

GDGoC TUM Campus Heilbronn · GenAI Builders Day · partner challenge by EcoComply · build with IBM Bob + Twilio. Now go build the radar. 🚀

Where to get current regulations (start here)

The challenge is to assess the portfolio against **current** EU requirements. This is a curated starting list of real, public sources — you don't have to use all of them. Pick one or two, go deep, and **cite the URL** for every gap you report.

Tip: prefer **official APIs / RSS / downloadable lists** over scraping rendered HTML — they're more stable and polite. Respect `robots.txt` and rate limits, cache what you pull, and record the access date. If a site is hard or Wi-Fi fails on the day, fall back to `regulatory_updates.json / feed/`.

Primary legislation

Source	What you'll find	Access tip
EUR-Lex — eur-lex.europa.eu	The legal texts themselves: RoHS, REACH, the Battery Regulation, PPWR, GPSR, RED, EMC, LVD, ESPR, Toy Safety, MDR, POPs. Consolidated versions show what's in force <i>now</i> .	Has a webservice / SPARQL “Cellar” API and per-document RSS. Search by CELEX number.
EU Official Journal (OJ) — eur-lex.europa.eu/oj	Newly published acts, corrigenda and amendments, by date.	Daily OJ has an RSS feed — good for “what changed recently”.

Chemicals (RoHS / REACH / CLP / POPs)

Source	What you'll find	Access tip
ECHA — echa.europa.eu	The SVHC Candidate List , Annex XVII restrictions, CLP, the restrictions roadmap. The single best source for “which substances are restricted now”.	Candidate List and restriction lists are downloadable (and searchable); ECHA publishes news + RSS.

Energy & ecodesign

Source	What you'll find	Access tip
EPREL — eprel.ec.europa.eu	The EU energy-label database for displays, lighting, power supplies, etc.	EPREL has a public API — query by product group.
Ecodesign / ESPR — commission.europa.eu (energy-efficient-products)	Product-group ecodesign measures and the Digital Product Passport roadmap.	Mostly HTML + linked PDFs; pair with the EUR-Lex text.

National producer / EPR registers (market-specific)

Source	What you'll find	Access tip
Germany — stiftung ear (ElektroG) — stiftung-ear.de	WEEE/EEE producer registration obligations for the German market.	Registration rules + registered-producer lookup.
France — ADEME / SYDEREP + AGEC repairability index — ademe.fr / data.ademe.fr (Other EU states have their own EPR registers — relevant when a company sells there.)	EPR obligations and the French repairability/durability index.	ADEME publishes open datasets.

Product safety & recalls

Source	What you'll find	Access tip
Safety Gate / RAPEX — ec.europa.eu/safety-gate-alerts	Weekly alerts on dangerous non-food products (great for GPSR-style risk signals).	Has a weekly alerts feed/export.

Standards (RED / EMC / LVD)

Source	What you'll find	Access tip
Harmonised standards lists — single-market-economy.ec.europa.eu	Which harmonised standards give “presumption of conformity” per directive (incl. RED cybersecurity EN 18031).	Published as HTML tables + OJ citations.
ETSI / CENELEC — etsi.org	Radio/EMC standards detail.	Standards search.

Which source for which rule (quick map)

- Restricted **substances** (RoHS/REACH/POPs) → **ECHA** + the EUR-Lex text.
- **Batteries** (passport, carbon footprint, removability, labelling) → **EUR-Lex** Reg (EU) 2023/1542.

- **Packaging** (PPWR) → **EUR-Lex** Reg (EU) 2025/40.
- **Radio** (common charger, cybersecurity) → **EUR-Lex** Directive 2014/53/EU + harmonised-standards list.
- **Energy label / ecodesign** → **EPREL** + ESPR pages.
- **WEEE / national EPR** → the relevant **national register** (DE, FR, ...) for the markets a company sells in.
- **General safety / recalls** → **Safety Gate** + GPSR text.

How to cite a gap

For every finding, record: the **requirement**, the **source URL** you read it from, the **deadline**, and which **company/product** it hits and why. That's what makes your result trustworthy to the jury — and to EcoComply.

Appendix — Data Dictionary

The field-by-field guide to the dataset. For the **challenge itself** (what to build, how to start, how you're judged) read **README.md**. For **where to get current regulations**, read **SOURCES.md**.

The company portfolio is your fixed dataset. Your job is to pull *current* EU requirements from live sources, find where these companies are not compliant today, and alert them. The portfolio companies and contacts are **fabricated and safe** - @example.com emails and placeholder phone numbers, so test alerts never reach a real person. Use your **own** Twilio test number for the demo.

What's in here

File	What it is
partners.json	22 synthetic SME companies and their 53 products - the portfolio you assess. A few carry illustrative current gaps (5 companies); the rest are for you to assess.
partners.csv	The same portfolio, one row per product, for spreadsheets.
taxonomy.json	Controlled vocabulary: product categories, substances, regulation families.
sample_expected_output.json	The shape of one finding (gap + source + alert) - what your agent should emit.
regulatory_updates.json	Offline sample / fallback feed (50 example updates). NOT the task - use it only to develop offline or if live scraping fails.
feed/	10 sample HTML notices to practise scraping/ingestion against, offline.
dataset_stats.json	Summary counts.

The portfolio (partners.json)

Each **partner** has `partner_id`, `company`, `hq_country`, `sells_in` (markets; EU = all 27 states), and a `contact` (name, email, phone, preferred_channel).

Each **product** carries the attributes you reason about to infer obligations: `category`, `substances`, `has_battery` + `battery_type` (portable/button_cell/lmt/industrial) + `battery_capacity_wh`, `has_radio`, `connector`, `packaging`, `intended_use` (consumer/toy/industrial/medical), `markets`, and `compliance_streams`.

A few companies also include a **compliance_status** block: `certs_held` and `known_gaps` (concrete current failings you can verify against live sources). Most companies have no such block on purpose - working out their gaps from current requirements is the challenge.

How to think about obligations & gaps

For each company/product, an obligation generally applies when the product's profile lands in a rule's scope: 1. **Market** - the company sells where the rule applies (EU = all 27 states; some rules are one country only). 2. **Category** - the rule covers the product's category (or all electronics). 3. **Substance** - if a rule names a substance, the product actually contains it. 4. **Attributes** - e.g. battery type/capacity, has-radio, connector, intended use, packaging. 5. **Exclusions** - e.g. general consumer-safety rules don't cover medical/industrial-only equipment.

A **gap** is an obligation that applies *and* the company hasn't met yet. Watch the look-alikes: right category but wrong market, a substance that isn't actually present, or an attribute that takes a product out of scope (e.g. a passport rule that hits industrial batteries but not portable ones). Cite the source you used for every gap.

The offline sample feed (regulatory_updates.json + feed/)

This is a **fallback**, not the task. The real signal is the live web (see SOURCES.md). The sample is deliberately messy - it mixes in unrelated-domain noise and duplicate/correction entries (`corrects` field) - so even offline you practise filtering and de-duplication. Each entry has a free-text `summary` plus a `scope` block (categories / substances / markets / conditions).

The portfolio is generated from a maintainer script and is intentionally synthetic.